

Technical Assessment

Technical Assessment

Production-Ready Containerized Inventory & Order Management System

Role

Software Engineer

1. Objective

Build a full-stack Inventory & Order Management System that allows businesses to manage:

- Products
- Customers
- Orders
- Inventory tracking

The application must include:

- A React frontend
- A Python backend API
- A PostgreSQL database

The entire system must be:

- Fully containerized using Docker
- Managed using Docker Compose
- Deployed online using free hosting platforms

2. Required Technology Stack

You must use the following technologies.

Backend

- Python
- FastAPI or Flask

Frontend

- React (JavaScript)

Database

- PostgreSQL

Containerization

- Docker

Service Orchestration

- Docker Compose

Version Control

- Git

3. Functional Requirements

3.1 Product Management

The system must support the following product operations.

APIs

POST /products

Create a new product.

GET /products

Retrieve all products.

GET /products/{id}

Retrieve a specific product by ID.

PUT /products/{id}

Update product details.

DELETE /products/{id}

Delete a product.

Product Fields

Each product should contain at minimum:

- Product name
- SKU/code
- Price
- Quantity in stock

3.2 Customer Management

The system must support customer management features.

APIs

POST /customers

Create a new customer.

GET /customers

Retrieve all customers.

GET /customers/{id}

Retrieve customer details by ID.

DELETE /customers/{id}

Delete a customer.

Customer Fields

Each customer should contain at minimum:

- Full name
- Email address
- Phone number

3.3 Order Management

The system must support order creation and tracking.

APIs

POST /orders

Create a new order.

GET /orders

Retrieve all orders.

GET /orders/{id}

Retrieve order details by ID.

DELETE /orders/{id}

Cancel/Delete an order.

Order Requirements

An order must include:

- Customer reference
- Product reference(s)
- Quantity ordered
- Total amount

4. Business Logic Requirements

Your application must implement the following rules.

- Product SKU/code must be unique.

- Customer email must be unique.
- Product quantity cannot be negative.
- Orders cannot be placed if inventory is insufficient.
- Creating an order must automatically reduce available stock.
- The total order amount must be calculated automatically by the backend.
- All APIs must include proper error handling.
- Use appropriate HTTP status codes.
- Validate all request data before processing.

5. Frontend Requirements

Build a responsive React application that integrates with the backend APIs.

Frontend Features

Product Management

- Add product
- View product list
- Update product
- Delete product

Customer Management

- Add customer
- View customer list
- Delete customer

Order Management

- Create order
- View orders
- View order details

Dashboard

Display summary information including:

- Total products
- Total customers
- Total orders

- Low stock products

6. UI/UX Requirements

The frontend application should meet the following requirements.

- Responsive design for desktop and mobile devices
- Clean and professional user interface
- Proper form validation
- Clear error and success messages
- Organized component structure
- Proper state management

7. Docker Requirements (Mandatory)

The project must be fully containerized.

Required Setup

You must provide:

- A production-ready Dockerfile for the backend
- A Dockerfile for the frontend
- A `.dockerignore` file
- Environment variable configuration
- A `docker-compose.yml` file

Docker Compose Services

The Docker Compose setup must run:

- Frontend service
- Backend service
- PostgreSQL database service

Additional Requirements

- Use slim/lightweight base images
- Do not hardcode credentials
- Use named volumes for PostgreSQL persistence

8. Deployment Requirements

The application must be deployed online using free hosting platforms.

Backend Deployment

Deploy the backend on one of the following:

- Render
- Railway
- Fly.io

Frontend Deployment

Deploy the frontend on one of the following:

- Vercel
- Netlify

Deployment Expectations

- Configure all environment variables correctly
 - Ensure frontend and backend communicate successfully
 - Ensure deployed URLs are publicly accessible and working
-

9. Submission Requirements

Submit the following items.

Required Deliverables

- GitHub repository link containing frontend and backend code

- Docker Hub image link for the backend image
- Live frontend deployment URL
- Live backend API URL